

Relatório Final

Metro Mondego System

Base de Dados

Ano Letivo 2025/2026 - PL8

Diogo Lemos nº 2020219666

Índice

Manual de Instalação	3
Manual de Utilizador	4
Authentication	4
SuperAdmin Endpoints	4
Admin Endpoints	4
Customer Endpoints	5
Analytics Endpoints	5
Decisões Tomadas	6
Arquitetura	6
Modelo de Dados	6
Transações e Concorrência	6
Autenticação	8
Performance	8
Decisões	9
Divulgação de uso de AI	9

Manual de instalação

Pré-requisitos

- Python 3.10 or higher
- PostgreSQL 14 or higher
- Postman

Database Setup

Criar a base de dados e o user no PostgreSQL:

```
CREATE USER metro_admin WITH PASSWORD 'metro_admin';  
CREATE DATABASE metro_db OWNER metro_admin;
```

Instalar dependencies

Correr na diretoria root do programa o comando:

```
pip install -r requirements.txt
```

Correr a API

Para correr o server com uma nova base de dados (drop e criação das tabelas, triggers e dados de teste), correr o comando:

```
python rest_api.py --setup
```

Para correr sem fazer esse reset (em testes consequentes):

```
python rest_api.py
```

A API estará disponível em <http://127.0.0.1:8080>.

Testes com o Postman

Fazer import no Postman da coleção metro_mondego.json em File→Import. A coleção contém todos os endpoints pré-configurados com exemplos de requests em body.

Manual de Utilizador

Authentication

1. User Authentication PUT /dbproj/user

Envia um username e uma password para receber uma JWT token. Esta token é automaticamente guardada na coleção e aplicada aos próximos pedidos.

```
{"username": "superadmin", "password": "superadmin2026"}
```

SuperAdmin Endpoint

2. Add Administrator PUT /dbproj/register/admin

Cria uma nova conta de administrador. É obrigatório estar autenticado como SuperAdmin.

```
{"name": "Admin Name", "username": "admin1", "email": "admin1@metro.pt", "password": "secret"}
```

Admin Endpoints

3. Add Customer POST /dbproj/register/customer

Cria um novo cliente.

```
{"name": "Customer Name", "username": "customer1", "nif": "123456789", "phone": "910000000", "email": "customer1@metro.pt", "password": "secret"}
```

4. Update Line Operation Settings PUT /dbproj/line_operation/{line_id}

Atualiza os dados operacionais da linha correspondente a {line_id}. Tem que ser transmitido pelo menos um campo para atualizar. Na coleção do Postman existe um exemplo de como se pode fechar uma linha.

```
{"start_time": "07:30:00", "end_time": "21:00:00", "frequency_minutes": 20, "vehicle_capacity": 50, "is_open": true}
```

5. Update Fare Price PUT /dbproj/fares/{fare_id}

Atualiza o preço de uma fare para bilhetes a partir de uma certa data. É adicionado um novo registo em fare_history para guardar os preços passados.

```
{"price": 2.75, "effective_from": "2026-06-01"}
```

6. Broadcast Notice POST /dbproj/notices/broadcast

Envia um aviso para todos os clientes registados simultaneamente.

```
{"title": "Strike Notice", "message": "Possible delays between 10:00 and 13:00"}
```

7. Create Promotion POST /dbproj/promotions

Cria uma promoção para uma certa linha e tipo de bilhete (diário/mensal). O product_type tem de ser single_trip, daily, monthly, monthly_student, monthly_senior. {"name": "School Holidays", "line_id": 1, "product_type": "daily", "discount_percent": 20, "start_date": "2026-07-01", "end_date": "2026-07-31"}

Customer Endpoints

8. List Lines and Upcoming Departures GET /dbproj/lines_next

Retorna todas as linhas com as suas horas de saída, direção e capacidade. Não requer body.

9. Add Wallet Funds POST /dbproj/wallet/topup

Adiciona fundos à carteira do cliente que se encontra autenticado. {"amount": 20.00, "payment_method": "card"}

10. Purchase Ticket or Pass POST /dbproj/purchase

Compra um bilhete ou um passe para o cliente autenticado. É debitado saldo da carteira automaticamente através de um trigger.

{"line_id": 1, "product_type": "single_trip", "travel_date": "2026-05-20"}

11. Validate / Use Ticket POST /dbproj/ticket/use/{ticket_id}

Regista o uso de um bilhete. O {ticket_id} corresponde ao purchase_id retornado pelo endpoint 10. Verifica o tempo de validade e que a estação pertence à linha correta.

{"used_at": "2026-05-20 08:20:00", "station_id": 2}

15. Get Notices GET /dbproj/notices

Retorna todos os avisos ao cliente autenticado, ordenados do mais recente ao mais antigo. Após este retorno, os avisos são automaticamente marcados como lidos. Não exige body.

Analytics Endpoints

12. Peak and Low Demand Periods GET /dbproj/report/demand

Para cada linha, retorna o intervalo horário com maior e menor número de validação de bilhetes. Não requer body.

13. Top Spenders by Line GET /dbproj/report/top_spenders

Retorna o cliente/clientes que mais dinheiro gastaram no último mês para cada linha. Não exige body.

14. Monthly Report GET /dbproj/report/monthly

Para cada linha e mês, retorna o número de clientes com pelo menos uma validação de bilhete e com mais de duas validações. Não precisa de body.

Decisões Tomadas

Arquitetura

O código python encontra-se na pasta src/ e encontra-se dividido em tarefas: authentication.py (trata da autenticação e do registo dos utilizadores), customer_endpoints.py (endpoints do cliente), admin_endpoints.py (endpoints do admin), report_endpoints.py (endpoints finais de análise para admins), global_functions.py (desde variáveis globais como códigos de erro, a setup da base de dados e funções helper de segurança), hashing.py (hash das palavras passe) e rest_api.py (ficheiro principal para correr a aplicação e gerir as rotas e permissões de cada endpoint).

Modelo de dados

A entidade line_lineoperation é uma fusão de line e lineoperation numa só tabela. Foi necessária uma vez que os parâmetros operacionais são sempre 1:1 com linha.

A entidade purchase serve como uma identidade unificada para todos os tipos de bilhete e de passe. Travel_date representa o dia de viagem para single_trip e daily e o mês de validade para passes mensais.

Em farehistory, é preservado o histórico completo de preços. Nunca é apagado um registo anterior, é sempre adicionado a cada interação.

A entidade trip_purchase só é preenchida para single_trip. Passes não reservam trip específica. Para além disso, a PK inclui os três campos (trip_trip_id, trip_line_lineoperation_line_id, purchase_purchase_id), pois é necessário para permitir múltiplos clientes na mesma trip.

Transações e concorrência

Todas as operações de escrita usam BEGIN, COMMIT e ROLLBACK e é feito ROLLBACK em todos os early returns dentro de transações abertas.

Para impedir que dois clientes comprem o último bilhete ao mesmo tempo, é feito SELECT FOR UPDATE em trip de modo a que a segunda transação esteja bloqueada, até que a primeira seja completa ou dê rollback. Para além disso, foi criado um trigger que dispara antes do UPDATE de booked_seats, em que quando booked_seats > capacity lança exception e aborta o UPDATE.

O problema de quando um cliente tenta fazer duas compras ao mesmo tempo, é resolvido também com SELECT wallet_balance FOR UPDATE. Para além disso, para garantir que o saldo não fique negativo, foram criados os triggers trg_deduct_wallet_on_purchase, que vai ler o saldo atual e verificar se é suficiente para final_price e debita e trg_check_wallet_balance. Se for insuficiente ou se wallet_balance < 0, lança exceção e a transação é abortada.

Para garantir que um admin não altera o preço de uma fare enquanto um cliente está a comprar um bilhete para esse fare, a query do purchase_ticket bloqueia a linha do preço durante toda a transação de compra com FOR UPDATE. Ou seja, os administradores ficam impedidos de inserir um novo registo em fare_history para o mesmo fare_type_id até a compra terminar.

Para impedir que um cliente tente usar o mesmo bilhete single_trip duas vezes em simultâneo, é feito lock na linha purchase para garantir que as duas transações são serializadas. Também foi criado um trigger trg_check_single_trip_validation disparado BEFORE INSERT ON ticketvalidation, que conta os registos existentes em ticket_validation para o mesmo purchase_purchase_id e se já existir pelo menos um lança exceção.

No que toca à inserção de validações e compras em simultâneo durante a geração de um relatório, os três endpoints em questão são inicializados com BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE. O nível SERIALIZABLE garante que qualquer escrita concorrente que não tenha feito COMMIT antes do BEGIN não é visível.

Autenticação

A autenticação é feita com JWT, com três roles (superadmin, admin e customer) verificados por @role_required. Cada token é válido por 2 horas.

As passwords são encriptadas com PBKDF2-HMAC-SHA256 com salt aleatório de 32 bytes e 100.000 iterações.

Performance

Foram criados índices nas colunas mais usadas em WHERE e JOIN.

```
CREATE INDEX idx_trip_departure ON trip(departure_time)
```

Usado em list_lines_next para filtrar as próximas partidas.

```
CREATE INDEX idx_trip_line ON trip(line_lineoperation_line_id)
```

Usado em JOIN com a linha em quase todos os endpoints .

```
CREATE INDEX idx_ticketvalidation_used ON ticketvalidation(used_at)
```

Usado nos 3 relatórios para agrupar por hora/mês.

```
CREATE INDEX idx_purchase_customer ON purchase(customer_users_user_id)
```

Usado no purchase_ticket e use_ticket para encontrar as compras do cliente.

```
CREATE INDEX idx_purchase_date ON purchase(purchased_at)
```

Usado no report_top_spenders para filtrar os últimos 30 dias.

```
CREATE INDEX idx_farehistory_effective ON farehistory(effective_from)
```

Usado em purchase_ticket para encontrar o preço mais recente.

Decisões

A validade do passe diário serve para 24 horas, não apenas para o dia da compra. Os passes mensais de estudante e sénior são tratados como passes mensais normais.

Um cliente não pode ter dois passes do mesmo tipo para o mesmo mês e linha. É feita a verificação no Python antes do INSERT.

Foi adicionado um Endpoint 15 – GET notices. Através deste endpoint, o cliente pode escolher ler todas as notices (ordenadas da mais recente para a mais antiga) e o read_at é atualizado na mesma transação de leitura.

Divulgação de uso de AI

Foram utilizadas ao longo do projeto ferramentas de AI. Para além de copiloto de auxílio à escrita de código, foi utilizado para gerar comentários e o ficheiro seed_data.sql.